

重庆邮电大学

实验报告册

学 年 学 期 : 2025 -2026 学 年 第 2 学 期

课 程 名 称 : 课程设计（硬件综合设计开发）

学 生 学 院 : 人工智能学院

专 业 : 计算机科学与技术

学 生 学 号 : 学号1 2023212270 学号2 2023212249 学号
3 2023212266

学 生 姓 名 : 姓名1 李程 姓名2 金文轩 姓名3王磊

联 系 方 式 : 手机号 13060226991

重庆邮电大学教务处制

组长姓名	李程	组长学号	2023212270	专业班级	04012305
团队成员	姓名	学号	任务分工	分工系数（百分比）	
	李程	2023212270	数码管以及环境搭建	33.3	
	金文轩	2023212249	红外遥控以及 UI 显示	33.3	
	王磊	2023212266	游戏主逻辑	33.3	
校内指导教师	曾素华	校外指导教师	无	本次实验学时数	20
实践日期	17 周	实践地点	综合实验楼 B509		
项目名称	方向：俄罗斯方块				
评阅人签字				成绩	
一、选题及内容					
题目：基于 ARM 嵌入式平台的红外遥控俄罗斯方块游戏开发 本课题以 ARM Linux 嵌入式开发板为硬件载体，设计并实现一款基于红外遥控交互的轻量化俄罗斯方块游戏系统。现阶段嵌入式实训项目大多仅针对单一外设驱动开展开发，同时融合图形界面渲染、人机交互、多线程并发调度的综合性实训案例相对稀缺。本项目整合交叉编译、红外外设驱动、图形界面绘制、游戏逻辑运算、线程同步控制等多项嵌入式核心技术，兼具实训学习价值与工程实践意义。 系统采用标准 C 语言模块化开发，整体拆分为游戏核心逻辑、UI 界面渲染、红外指令解析、计分统计四大独立模块。工程通过`arm-none-linux-gnueabi`交叉编译工具链完成跨平台编译，配套 Makefile 脚本实现自动化构建；编译配置 gnu99 标准适配现代 C 语法，链接 pthread 多线程库，分离画面刷新、红外按键检测任务，实现并发运行。硬件搭载红外接收模块，解析遥控器移动、旋转、下落、暂停等操作指令；软件完整实现七类方块随机生成、碰撞边界校验、方块旋转、满行消除计分、游戏结束判定等核心玩法，实时刷新分数与游戏状态界面。 项目开发覆盖嵌入式工程搭建、交叉编译、根文件系统部署、多线程同步、外设驱动对接等核心实操内容，改善嵌入式设备交互形式单一、手动编译流程复杂等痛点。区别于 PC 端游戏程序，本方案适配嵌入式硬件资源有限的运行环境，对内存占用、画面刷新速率进行轻量化优化。 本系统可作为嵌入式 Linux 综合实训成果，完整串联底层外设驱动、上层业务逻辑与工程自动化编译全流程，既能夯实 C 语言编程、Linux 系统开发基础，也可为同类嵌入式小型交互式应用提供成熟、可复用的开发框架与实现思路。					
二、需求分析					

一、功能需求

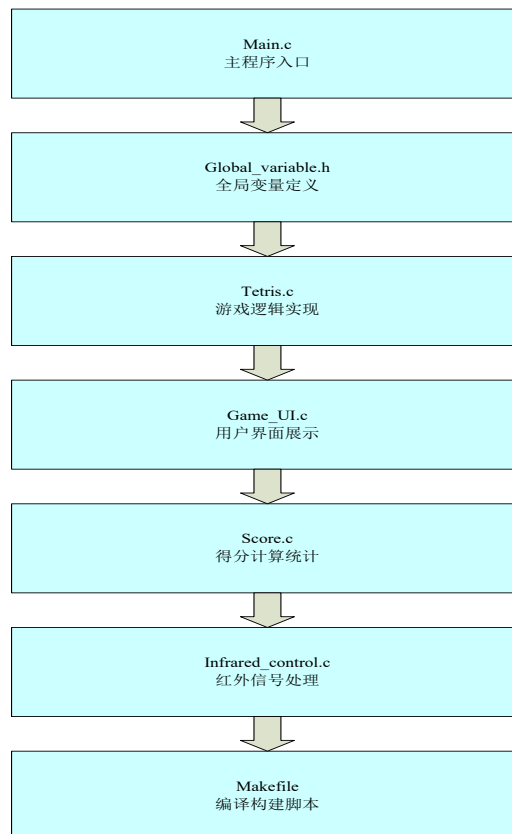
- 1. 游戏逻辑功能：程序可随机生成七种标准俄罗斯方块，支持方块左移、右移、加速下落、形态旋转；实时完成边界、方块堆叠碰撞检测，整行填满后自动消除并累计得分，方块堆叠溢出游戏区域顶部则判定游戏结束。
- 2. 红外交互功能：外接红外接收外设，识别遥控器操作指令，实现方块控制、游戏暂停、重新开始等操作，终端界面实时展示当前分数与游戏运行状态。
- 3. 界面渲染功能：在嵌入式显示终端绘制游戏网格、动态下落方块与堆积方块，同步展示计分信息，画面刷新流畅，无明显卡顿、闪烁问题。
- 4. 工程编译部署功能：采用 ARM 交叉编译工具链，通过 Makefile 实现一键编译构建；编译成功后自动将可执行程序迁移至开发板根文件系统，配套清理命令删除编译中间文件。
- 5. 多线程并发功能：使用 pthread 多线程分离界面刷新、红外按键检测任务，解决单线程阻塞导致的按键响应滞后、画面冻结问题。

二、非功能需求

- 1. 环境适配性：基于 ARM Linux 硬件平台开发，代码遵循 gnu99 标准，程序轻量化设计，适配嵌入式设备有限的内存与运算资源。
- 2. 系统稳定性：长时间连续运行无内存泄漏、程序崩溃；红外信号识别精准，不存在指令丢失、误触发等异常。
- 3. 使用便捷性：编译流程简单，无需复杂环境配置；游戏操作逻辑简洁直观，遥控器按键对应功能清晰，上手门槛低。

三、功能设计（框图、流程图等）

一、俄罗斯方块项目功能结构框图
采用层级化结构展示模块划分



功能结构说明：

核心层：tetris.c 是核心逻辑模块，负责方块的生成、移动、旋转、行消除、游戏结束判定等核心规则；

交互层：

infrared_control.c: 接收红外遥控信号，转换为游戏操作指令（上 / 下 / 左 / 右 / 旋转）；

game_UI.c: 将游戏状态（当前方块、得分、游戏等级）渲染到显示设备；

数据层：global_variable.h 定义全局变量，统一管理游戏状态（如当前分数、方块坐标、游戏是否运行）、硬件状态等，供所有模块调用；

辅助层：score.c 基于核心逻辑的消除行数计算分数、升级等级，支持最高分存储 / 读取；

构建层：Makefile 负责整个项目的编译、链接，生成可执行文件。

二、俄罗斯方块项目设计流程图

涵盖从初始化到游戏循环的完整流程。

设计流程说明：

初始化阶段：完成硬件（显示、红外接收）和软件（全局变量、UI）的初始化，为游戏运行做准备；

方块生成阶段：随机生成初始方块，确定初始位置（通常在顶部中间）；

主循环阶段：

优先检测红外遥控输入，处理玩家的主动操作（移动 / 旋转），并校验操作合法性；

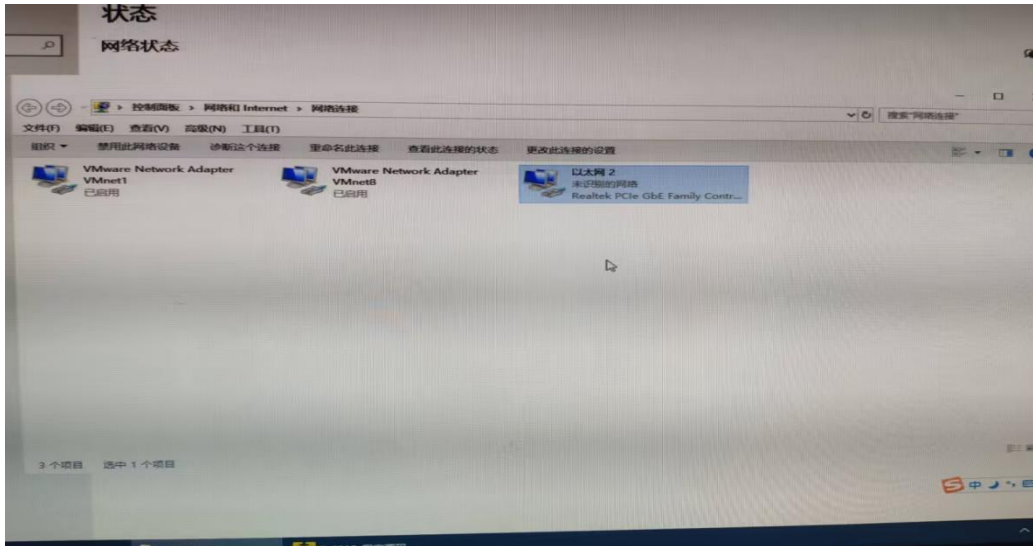
无输入时，方块按固定时间步自动下落；

检测方块触底 / 碰撞后，固定方块并检测消除行，更新分数和等级；

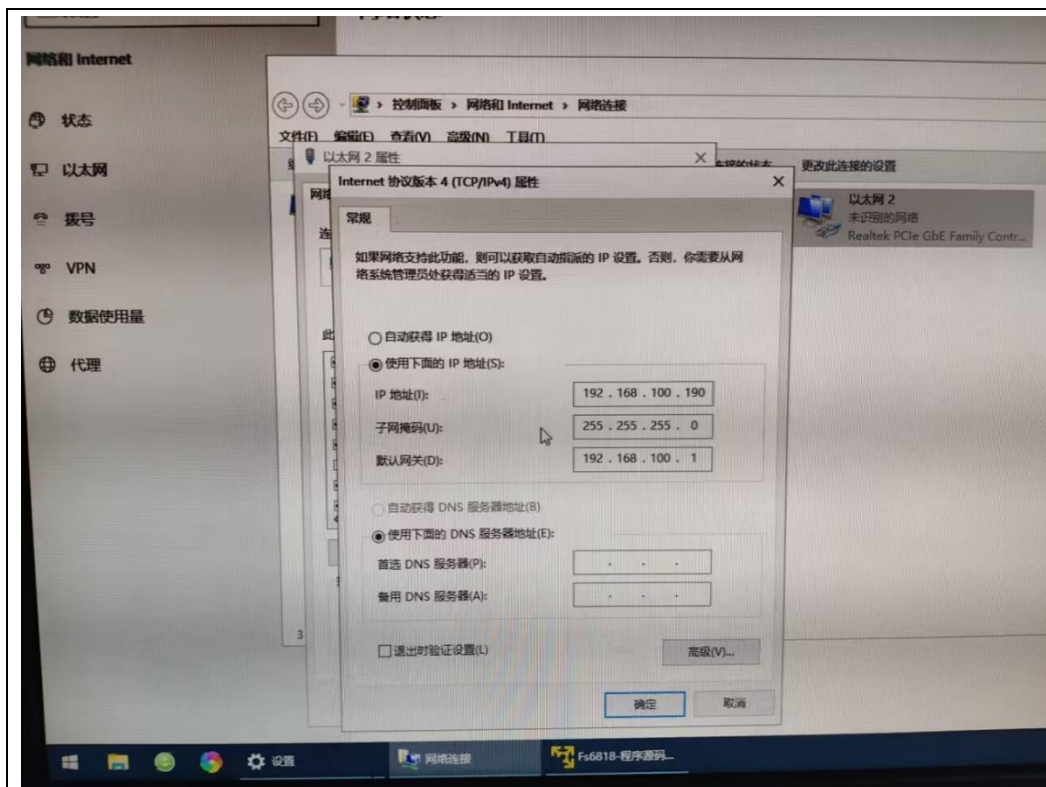
终止判定：新生成的方块若直接与已有方块 / 边界碰撞，判定游戏结束，否则循环生成新方块继续游戏。

四、开发流程步骤（实际过程的操作步骤，包括各种环境安装与配置、编译和交叉编译、程序下载或烧写、运行看结果等的阐述）

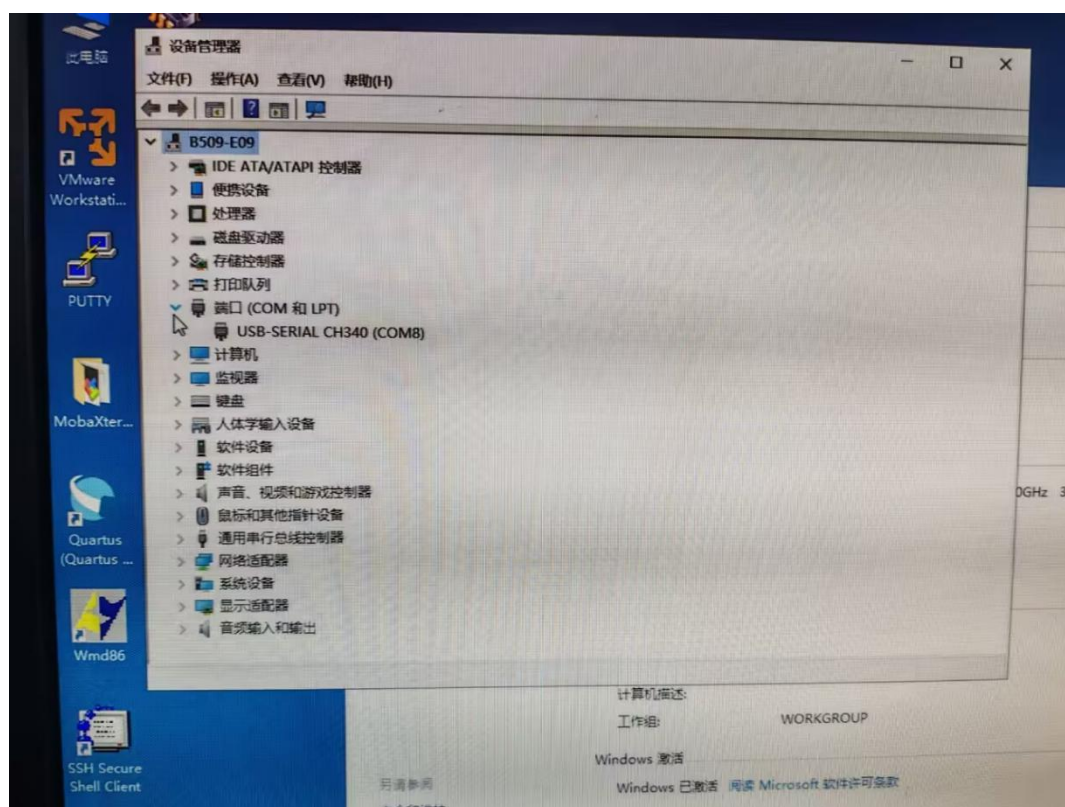
1. 打开网络和 internet 设置，点击更改适配器选项，进入网络连接界面，打开 VMNET1 和 8。



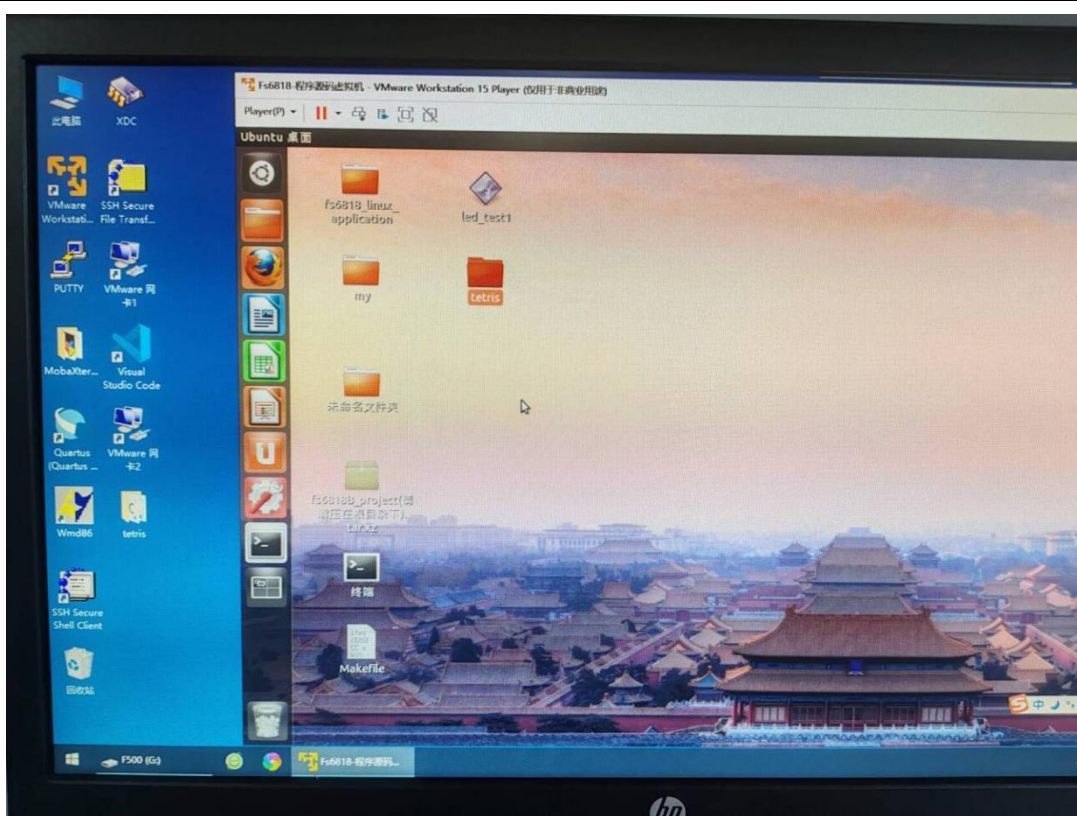
2. 配置以太网 IP 地址



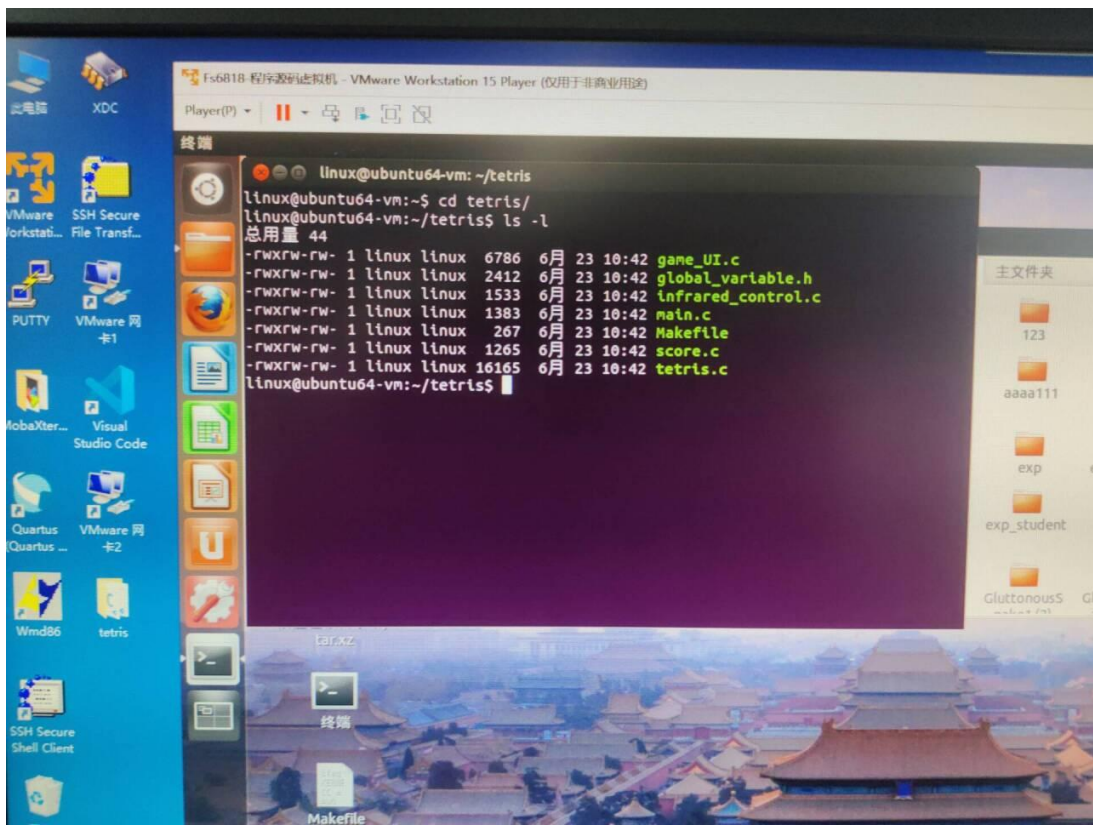
3. 打开设备 COM8 管理器查看 COM 是否成功连上

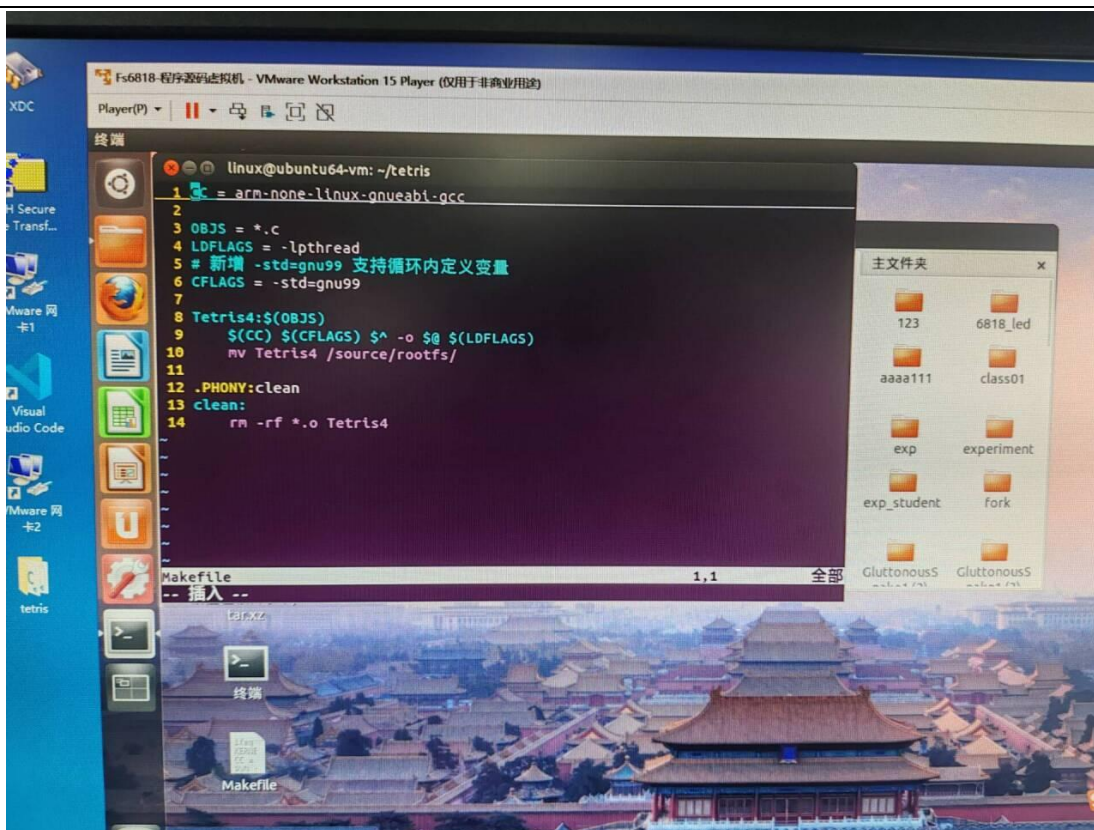


4. 将做好的项目保存到主机桌面上，移到虚拟机桌面上

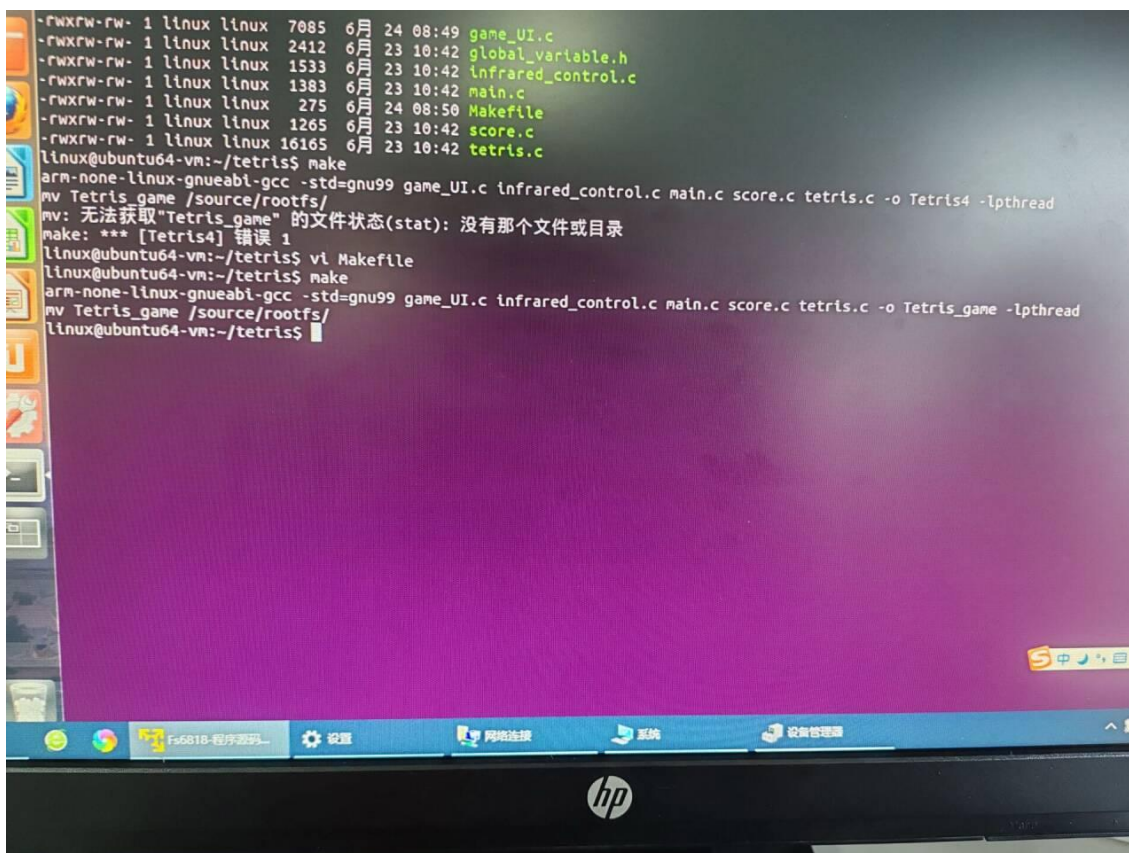


5. 进入项目目录里，查看 Makefile 文件

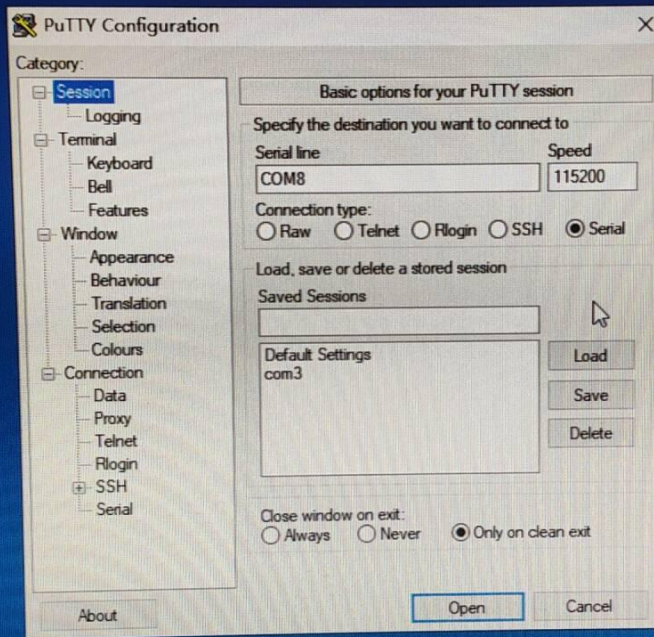




6. Make, 然后自动将程序移动到共享文件系统目录 /source/rootfs/



7. 打开 putty, 配置相关信息



8. 运行 Tetris_game 可执行文件，开始游戏

```

linux linux 2412 6月 23 10:42 global_variable.h
linux linux 1533 6月 23 10:42 infrared_control.c
linux linux 1383 6月 23 10:42 main.c
linux linux 275 6月 24 08:50 Makefile
linux linux 1265 6月 23 10:42 score.c
linux linux 16165 6月 23 10:42 tetris.c
64-vm:~/tetris$ v
64-vm:~/tetris$ COM8 - PuTTY
ux-gnueabi-gcc -Sdrwxrwxr-x 2 1000 1000 4096 Dec 15 2021 zlg7190
ne /source/rootfsdrwxrwxr-x 2 1000 1000 4096 Dec 17 2021 zlg72906
64-vm:~/tetris$ v[root@farsight ]# ./Tetris_game
64-vm:~/tetris$ v
64-vm:~/tetris$ r
ux-gnueabi-gcc -S
source/rootfs/
64-vm:~/tetris$ l
linux linux 7081
linux linux 2411
linux linux 1531
linux linux 1381
linux linux 261
linux linux 1261
linux linux 1616
u64-vm:~/tetris$ v
u64-vm:~/tetris$ v
nux-gnueabi-gcc -S-- 红外线程启动成功 --
/source/rootfs -- 计分系统启动成功 --
u64-vm:~/tetris$ l-- 俄罗斯方块线程启动成功 --

TETRIS

--按下红外遥控器'1'~'9'选择难度--
--难度将在数码管右半边显示--

--by 王磊、金文轩、李程

1 linux linux 7087 6月 24 09:18 game_UI.c
1 linux linux 2412 6月 23 10:42 global_variable.h
1 linux linux 1533 6月 23 10:42 infrared_control.c
1 linux linux 1383 6月 23 10:42 main.c
1 linux linux 269 6月 24 09:22 Makefile

```

```
linux linux 1533 6月 23 10:42 infrared_control.c
linux linux 1383 6月 23 10:42 main.c
linux linux 275 6月 24 08:50 Makefile
linux linux 1265 6月 23 10:42 score.c
linux linux 16165 6月 23 10:42 tetris.c
4-vm:~/tetris$ v
4-vm:~/tetris$ r COM8 - PuTTY
x-gnueabi-gcc -s [动画提示] 检测到选择难度，开场动画已关闭。
e /source/rootfs
4-vm:~/tetris$ v
4-vm:~/tetris$ v
4-vm:~/tetris$ r
x-gnueabi-gcc -s
source/rootfs/
4-vm:~/tetris$ l

linux linux 7087 6月 24 09:18 game_UI.c
linux linux 2412 6月 23 10:42 global_variable.h
linux linux 1533 6月 23 10:42 infrared_control.c
linux linux 1383 6月 23 10:42 main.c
linux linux 269 6月 24 09:22 Makefile
linux linux 1265 6月 23 10:42 score.c
linux linux 16165 6月 23 10:42 tetris.c

64-vm:~/tetris$ v
64-vm:~/tetris$ r
ux-gnueabi-gcc -s
/source/rootfs
64-vm:~/tetris$ l
```

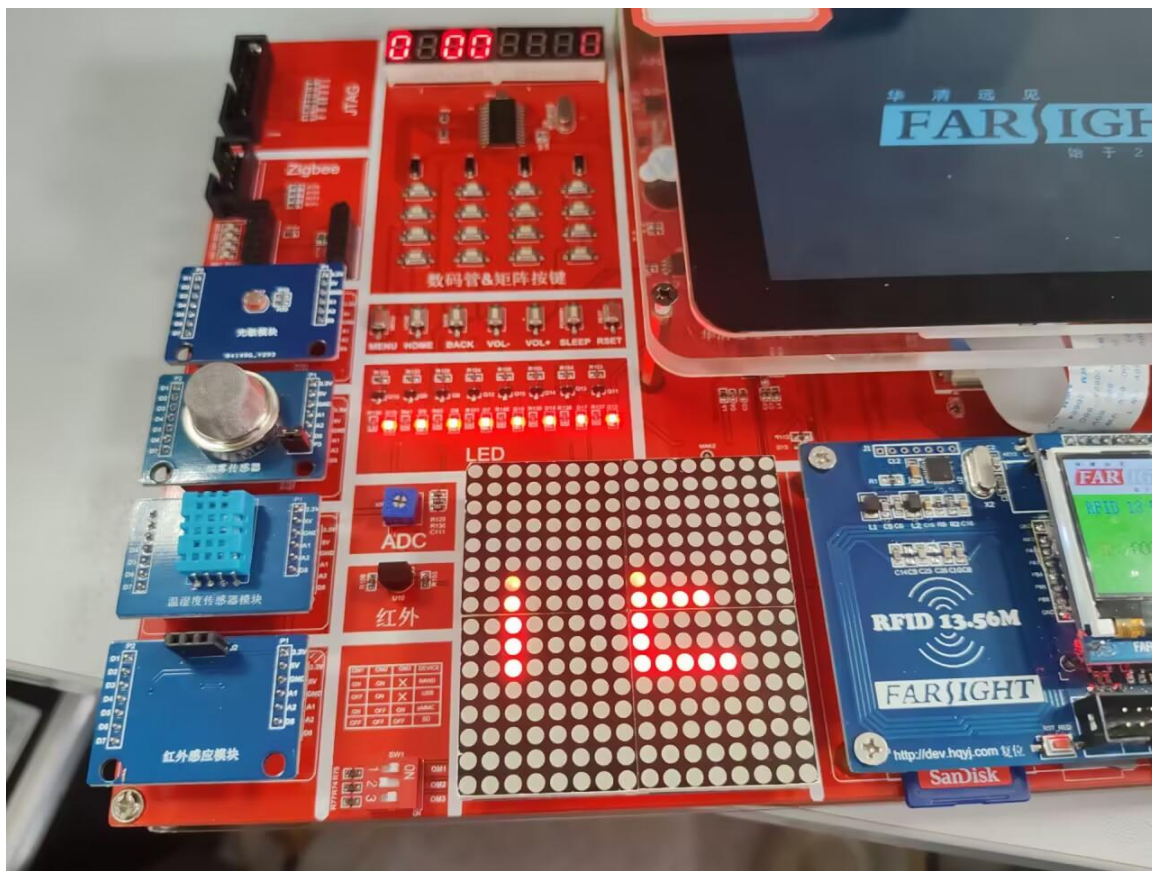
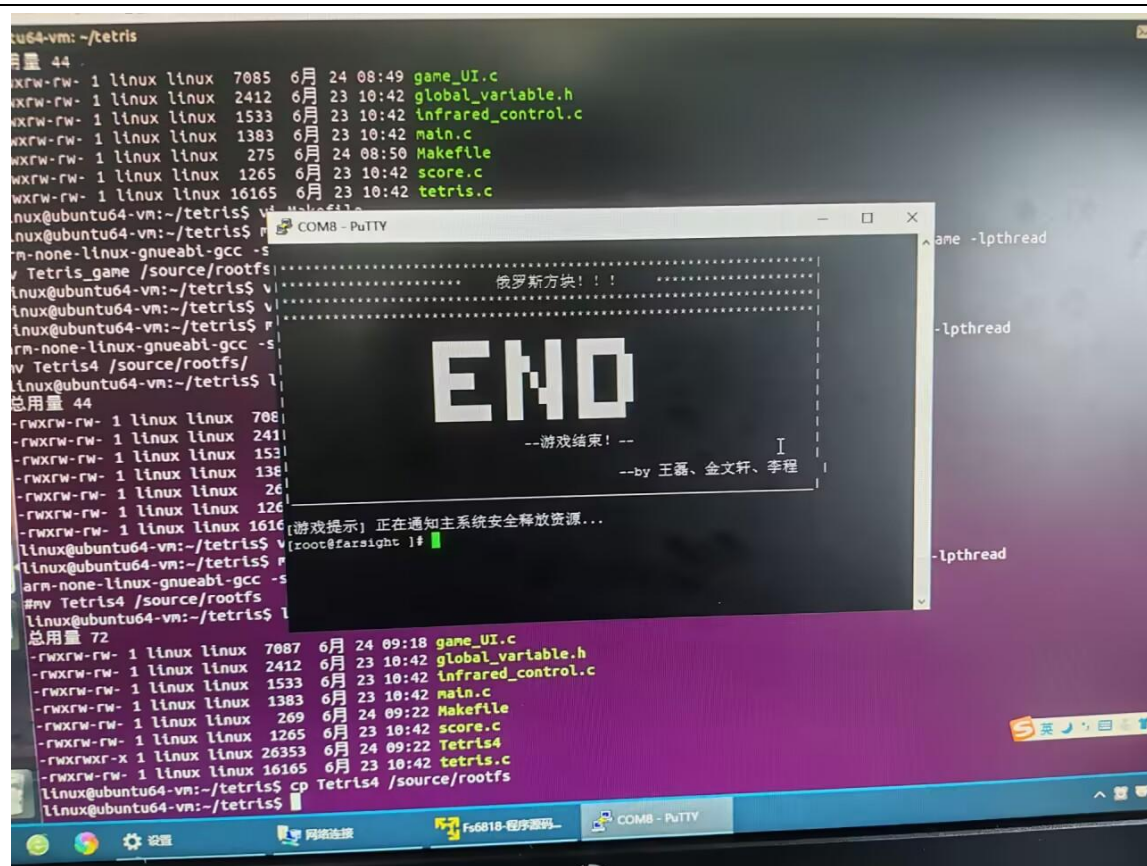


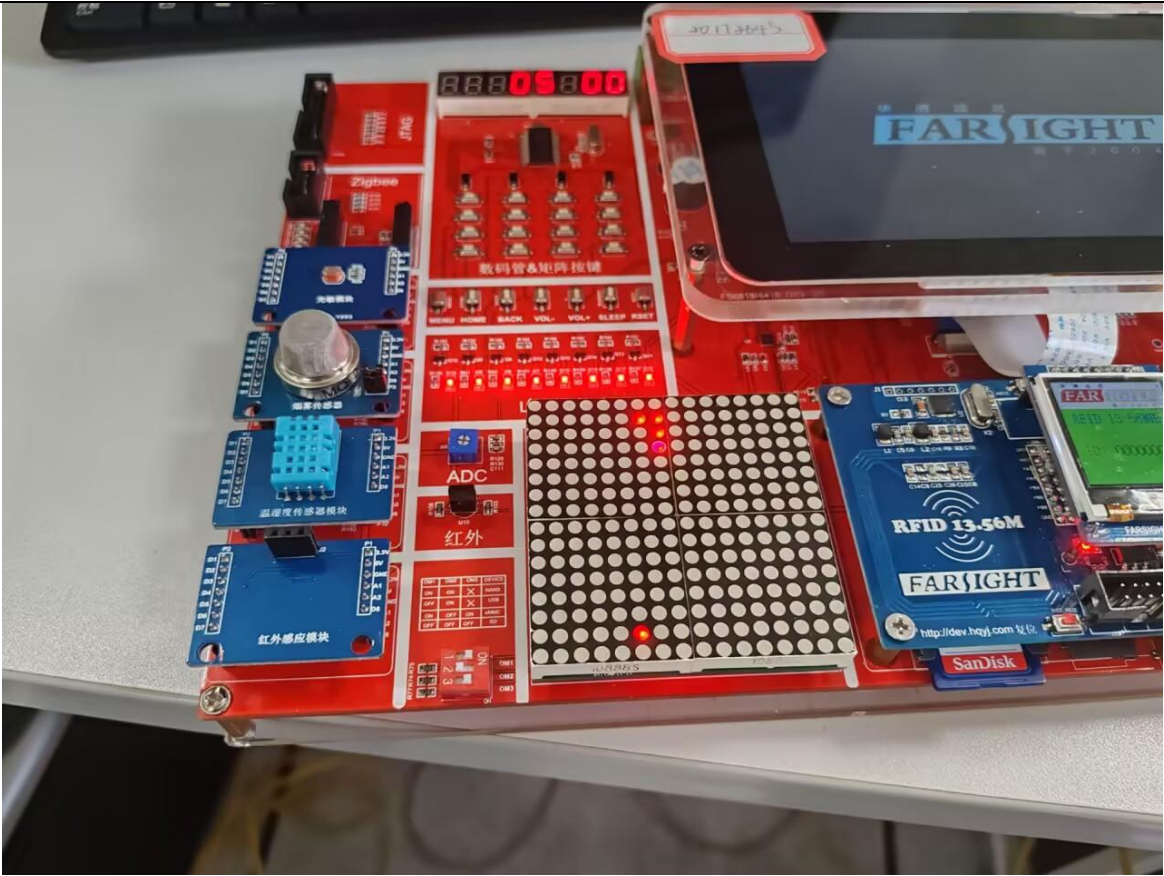
```
ux linux 1533 6月 23 10:42 infrared_control.c
ux linux 1383 6月 23 10:42 main.c
ux linux 275 6月 24 08:50 Makefile
ux linux 1265 6月 23 10:42 score.c
ux linux 16165 6月 23 10:42 tetris.c
vm:~/tetris$ v
vm:~/tetris$ r COM8 - PuTTY
gnueabi-gcc -s
/source/rootfs
-vm:~/tetris$ v
-vm:~/tetris$ v
-vm:~/tetris$ r
-gnueabi-gcc -s
source/rootfs/
-vm:~/tetris$ l

linux linux 7087 6月 24 09:18 game_UI.c
linux linux 2412 6月 23 10:42 global_variable.h
linux linux 1533 6月 23 10:42 infrared_control.c
linux linux 1383 6月 23 10:42 main.c
linux linux 269 6月 24 09:22 Makefile
linux linux 1265 6月 23 10:42 score.c
linux linux 16165 6月 23 10:42 tetris.c

64-vm:~/tetris$ v
64-vm:~/tetris$ r
ux-gnueabi-gcc -s
/source/rootfs
u64-vm:~/tetris$ l
```



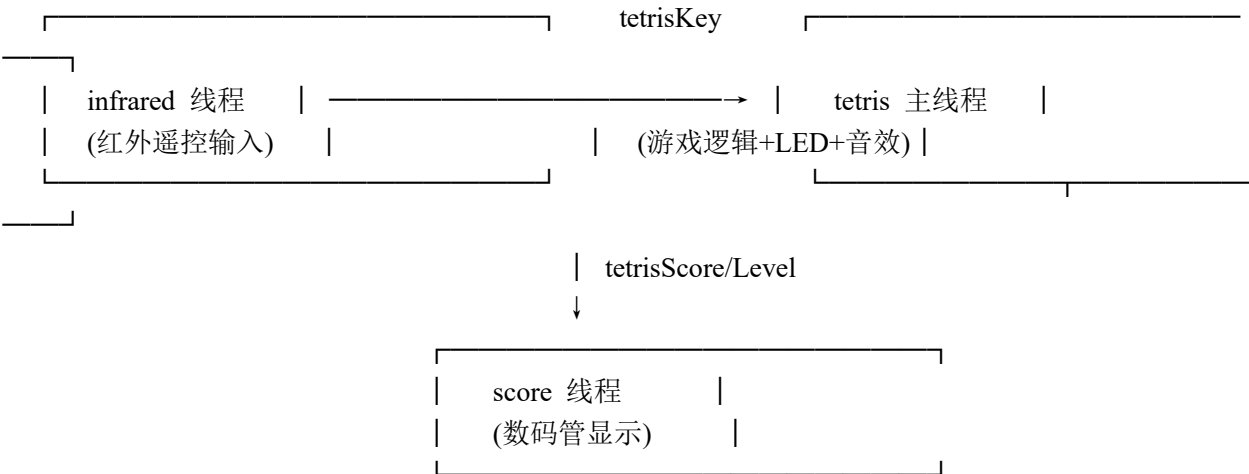




五、详细设计（核心代码+说明）

一、系统架构

三个并发线程通过全局变量通信：



二、核心数据结构（global_variable.h）

=====

=

// 方块结构体：4×4 矩阵描述形状，x/y 为在地图上的坐标

```

typedef struct {
    int x, y;
    int shape[4][4];
    int type;      // 0=I, 1=O, 2=T, 3=L, 4=J, 5=S, 6=Z
    int rotate;    // 0~3 四种旋转形态
} TetrisBlock;

// 16×16 LED 点阵显存：每 2 字节对应一行 16 列
// code[2*y] 存高 8 列，code[2*y+1] 存低 8 列
struct led1616Data {
    unsigned char code[33];
};

// 全局变量（线程间通信，每个变量只有一个写入者，无需锁）
char tetrisKey;      // 红外→游戏：当前按键
int  tetrisScore;    // 游戏→计分：分数
int  tetrisLevel;    // 游戏→计分：难度
int  tetrisSpeed;    // 由难度计算：speed = (10-level)*100ms

```

三、游戏主逻辑（tetris.c）

=

3.1 七种方块的旋转矩阵

```

// blockShapes[类型][旋转][行][列] — 7 种×4 旋转×4×4
int blockShapes[7][4][4][4] = {
    // I 型
    // O 型
    // T 型, L 型, J 型, S 型, Z 型 ... 共 7×4×16 = 448 个预定义数据
};

```

3.2 开场动画：LED 点阵滚动 "TETRIS"

```

/*
 * 用 16 字节纵向字模（上下各空 4 行居中），在 16×16 LED 上滚动显示 5 个字母。
 * 每 50ms 偏移一列，第 7 列留空做字间距，offset 到底后循环。
 * 期间检测 tetrisKey，按下 1~9 立即退出进入游戏。
 */
static const unsigned char font_T[16] = {
    0,0,0,0,          // 上空 4 行
    0x7C,0x10,0x10,0x10,0x10,0x10,0x10,0x00, // 竖柱
    0,0,0,0
};
// font_E, font_R, font_I, font_S 同理（各 16 字节）

```



```

void tetris_opening_animation() {
    int offset = 0;
    while (1) {
        if (tetrisKey >= 'I' && tetrisKey <= '9') break; // 按键退出

        // 遍历 16×16 视窗，按 offset 偏移量将 5 个字符渲染到点阵
        for (y:0..15, x:0..15) {
            int global_x = offset + x - 16; // 文字从右滚入
            if (global_x ∈ [0, 48)) {
                int char_idx = global_x / 8; // 第几个字符
                int bit_idx = 7 - (global_x % 8); // 字模位序（高位在左）
                if ((global_x % 8) < 7) // 第 7 列为间距，跳过
                    pixel = (font[char_idx][y] >> bit_idx) & 1;
            }
            // 映射到硬件坐标: real_x = 15 - x（水平镜像）
            // real_x < 8 → code[2*y+1]; real_x ≥ 8 → code[2*y]
        }
        ioctl(dev_led1616_fd, LED_DISPLAY, &data);
        usleep(50000); offset++;
        if (offset >= 80) offset = 0; // 循环
    }
}

```

3.3 游戏主循环

```

void *tetris_thread(void *arg) {
    // 1. 打开设备: LED 点阵 + 蜂鸣器
    // 2. 播放开场动画 → 截获难度键 1~9 → 计算 tetrisSpeed = (10-level)*100ms
    // 3. 等待 OK 键 → 显示操作说明 → 初始化地图 + 随机生成首个方块

    while (1) {
        // ---- 按键处理 ----
        if (tetrisKey != 0) {
            // 在临时变量上试操作（L 左/R 右/D 下/U 旋转），碰撞检测通过才更新
            if (!check_collision(tempBlock))
                currentBlock = tempBlock; // 移动成功
            tetrisKey = 0; // 消费按键
        }

        // ---- 自动下落 ----
        usleep(tetrisSpeed);
        downBlock.y += 1;
        if (check_collision(downBlock)) {
            if (currentBlock.y < 0) { tetris_game_over(); return; } // 顶部溢出
            merge_block_to_map(currentBlock); // 固定方块到地图
            int lines = clear_full_lines(); // 消行
            if (lines) {

```

```

        tetrisScore += lines * 100;           // 加分
        if (tetrisScore/1000 >= tetrisLevel && tetrisLevel < 9)
            { tetrisLevel++; tetrisSpeed = (10-tetrisLevel)*100000; }
    }
    currentBlock = create_block();           // 下一块
} else {
    currentBlock = downBlock;               // 继续下落
}
tetris_print(); // 刷新 LED
}
}

```

3.4 碰撞检测（核心算法）

```

int check_collision(TetrisBlock block) {
    for (i:0..3, j:0..3)
        if (block.shape[i][j] == 1) {
            int x = block.x + j, y = block.y + i;
            if (x < 0 || x >= 16) return 1; // 左右墙
            if (y >= 16) return 1; // 底部
            if (y >= 0 && gameMap[y][x] == 1) return 1; // 与已固定方块重叠
        }
    return 0; // 无碰撞
}
// 注意: y<0 不判碰撞, 允许方块从顶部逐步落入

```

3.5 消行算法（从底部向上扫描 + 回退检查）

```

int clear_full_lines() {
    for (i = 15; i >= 0; i--) { // 从底部向上
        if (第 i 行全为 1) {
            for (k = i; k > 0; k--) // 上方所有行整体下移
                memcpy(gameMap[k], gameMap[k-1], 16*sizeof(int));
            memset(gameMap[0], 0, ...); // 顶部清零
            i++; // 回退: 下移后当前行变为新内容, 需重新检查
        }
    }
}

```

3.6 LED 渲染: 将地图+方块写入点阵显存, 通过 ioctl 刷新

```

void tetris_print() {
    memset(data.code, 0, 33);
    // 遍历 gameMap[y][x]==1 的像素 + currentBlock 的 shape 像素
    // 水平镜像: real_x = 15 - logic_x
    // real_x < 8 → code[2*y+1] |= (1 << real_x) (低 8 列)
    // real_x ≥ 8 → code[2*y] |= (1 << (real_x-8)) (高 8 列)
}

```

```

    ioctl(dev_led1616_fd, LED_DISPLAY, &data);
}

```

3.7 游戏结束：五音旋律 + 画面闪烁 + 资源释放

```

void tetris_game_over() {
    pthread_cancel(id_score_thread);          // 停止计分
    pthread_cancel(id_infrared_control_thread); // 停止红外
    // G→A→F→D→低 F 下行旋律 (880,698,587,494,349 Hz)，偶数拍画面闪烁
    // 最后 close 设备 → raise(SIGINT) → ReleaseResource() 回收全部资源
}

```

四、红外遥控输入 (infrared_control.c)

```

=====
=

// 键值映射表：硬件码 → 逻辑字符
key1_t table[] = {
    {KEY_1,'1'},...{KEY_9,'9'},          // 数字→难度
    {KEY_UP,'U'}, {KEY_DOWN,'D'},        // 方向→旋转/下落
    {KEY_LEFT,'L'}, {KEY_RIGHT,'R'},      // 方向→移动
    {KEY_ENTER,'O'},                     // OK→确认
};

void *infrared_control_thread(void *arg) {
    dev_ir_fd = open("/dev/input/event1", O_RDONLY | O_NONBLOCK); // 非阻塞
    while (1) {
        ret = read(dev_ir_fd, &ev, sizeof(input_event));
        if (ret <= 0) { usleep(2000); continue; } // 无数据休眠 2ms
        if (ev.type == EV_KEY && ev.value == 1)    // 仅处理按下事件(去抖)
            tetrisKey = 查表(ev.code);             // 写入全局变量
    }
}

// 设计要点: O_NONBLOCK 非阻塞 + ev.value==1 天然去抖

```

五、数码管显示 (score.c)

```

=====
=

void *score_thread(void *arg) {
    char buf[8] = "00000000";
    while (1) {
        itoa(tetrisScore, buf, 10);          // 分数→前 4 字节
        itoa(tetrisLevel, buf+4, 10);        // 难度→后 4 字节
        ioctl(dev_zlg7290_fd, SET_VAL, buf); // 刷新数码管
    }
}

```



```
        usleep(100000);                // 100ms 周期
    }
}
```

六、程序入口（main.c）

```
=====

int main() {
    start_interface1(); // 终端显示难度选择界面

    pthread_create(红外线程);
    pthread_create(计分线程);
    pthread_create(游戏主线程);

    signal(SIGINT, ReleaseResource); // 注册 Ctrl+C / 异常退出处理

    // 主线程阻塞等待子线程结束
    pthread_join(红外线程); pthread_join(计分线程); pthread_join(游戏线程);
}

void ReleaseResource() {
    pthread_cancel(所有线程);
    close(所有设备 fd: LED/数码管/红外/蜂鸣器);
    exit(0);
}
```

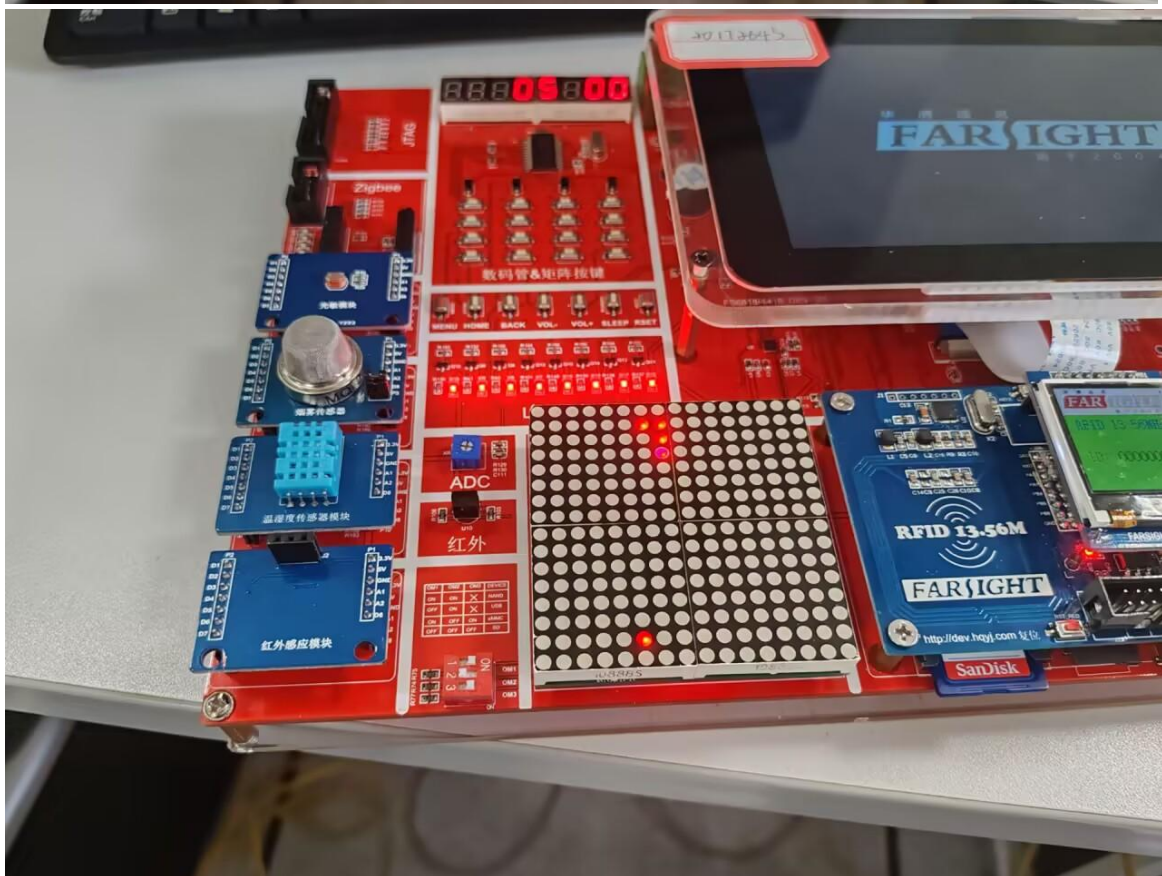
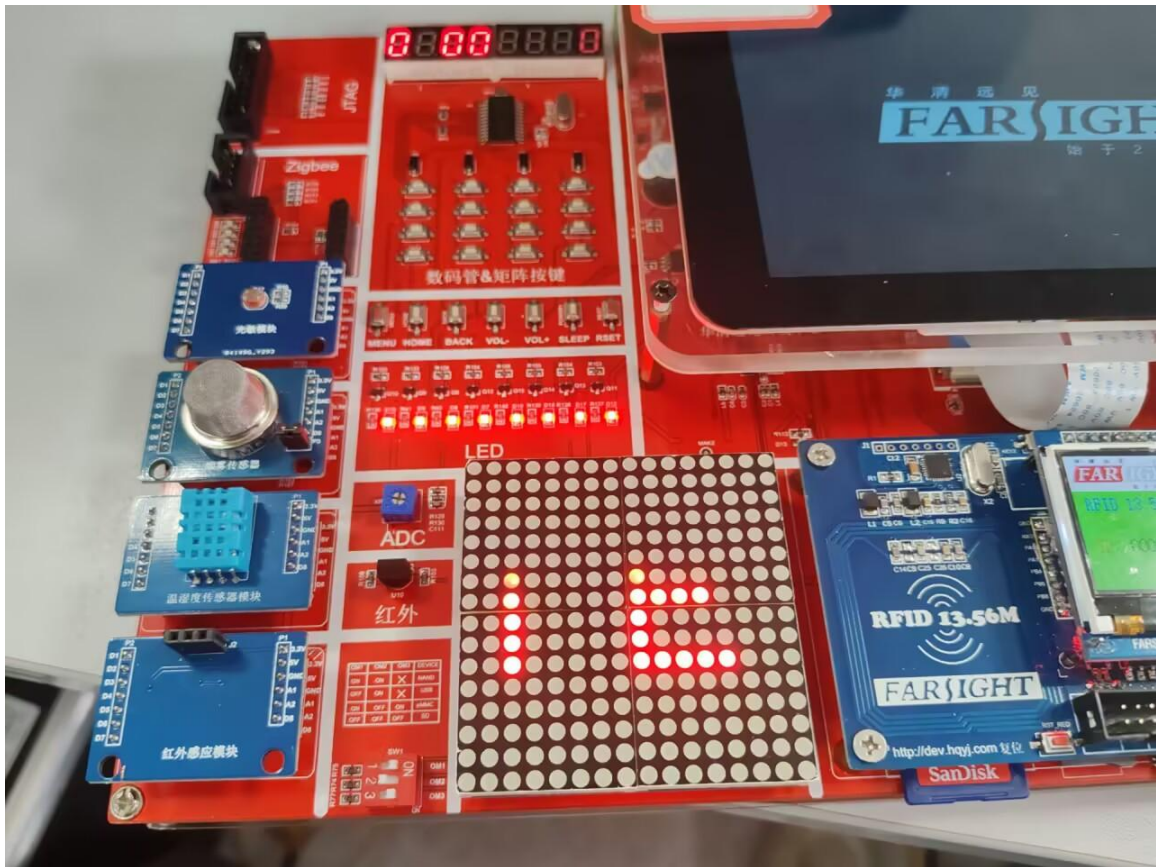
关键设计总结

```
=====

多线程并发    — 三个线程各司其职，独立运行不阻塞
全局变量通信  — tetrisKey→游戏, tetrisScore/Level→计分，每变量单写入者无锁
非阻塞 I/O    — 红外 O_NONBLOCK + 2ms 轮询，不影响游戏性能
碰撞预检测    — 先对临时变量试操作，检测通过才更新，避免非法状态
消行回退      — clear_full_lines() 中 i++ 确保下移后的新行被重新检查
硬件抽象      — 红外键码表映射，游戏逻辑不依赖具体硬件键值
难度线性加速  — speed = (10-level)×100ms，等级 1=900ms→等级 9=100ms
信号安全退出  — SIGINT → 取消线程 + 关闭所有 fd，保证资源不泄漏
```

六、系统实现（结果图）

效果运行截图：



七、总结

1、遇到问题及解决方案

1>. 多线程编程实践

掌握了 `pthread_create/pthread_join/pthread_cancel` 的用法，
理解线程间通过全局变量通信的模式及其适用场景（单写入者无需锁）

2>. Linux 设备驱动接口

接触了三种典型的外设交互方式：

- `ioctl`: LED 点阵 (`LED_DISPLAY`)、数码管 (`SET_VAL`)
- `read/input_event`: 红外遥控器 (`input` 子系统)
- `write/sysfs`: PWM 蜂鸣器 ("频率,占空比"字符串控制)

3>. 交叉编译与嵌入式部署

使用 `arm-none-linux-gnueabi-gcc` 交叉编译工具链，
编写 `Makefile` 自动将产物部署到 `/source/rootfs/` 目标板文件系统

4>. 实时游戏循环设计

理解了固定帧率（开场动画 20fps）与变速控制（游戏下落速度随难度变化）
的区别和实现方式

5>. 硬件抽象思维

通过键值映射表将硬件键码与游戏逻辑解耦，更换遥控器只需修改映射表

6>. 资源管理意识

信号处理 + 统一清理函数，确保异常退出时设备资源不泄漏

2、收获与发现不足或有待提高的地方:

1. 按键处理为单字符变量，无缓冲队列

当前 `tetrisKey` 一次只能存一个按键。如果玩家在游戏线程处理完之前
快速按下另一个键，第二个按键会覆盖第一个，造成"丢键"。
改进: 可使用环形缓冲区队列暂存按键序列

2. 多处忙等待（`busy-wait`）

如 `while(tetrisKey != 'O') usleep(10000)` 等轮询等待，
虽然加了休眠但本质上仍是 CPU 空转。
改进: 使用条件变量(`pthread_cond`)或信号量，让线程真正阻塞等待

3. 线程取消方式较为粗暴

`tetris_game_over()` 中直接 `pthread_cancel` 计分和红外线程，被取消的线程没有机会做清理工作。

改进: 使用标志位(如 `volatile int running=1`)通知线程自行退出，在线程内检查标志位并 `return`，资源由各线程自行释放

4. 缺少暂停/恢复功能的完整实现

代码中提到 OK 键可暂停/继续，但主循环中未见明显的暂停逻辑（仅有等待 OK 键开始的阶段）。暂停期间的画面保持、下落计时器冻结等功能未体现。

改进: 增加 `pause` 状态变量，暂停时跳过下落逻辑但保持画面刷新

5. 错误处理不够完善

设备 `open` 失败直接 `return NULL`，缺少重试或降级策略；游戏中途设备异常断开无检测机制。

改进: 对关键 `ioctl/read/write` 返回值做检查，失败时尝试恢复或安全退出

6. 自定义 `itoa()` 可替换

`score.c` 中手动实现了 `itoa()` 函数，而标准 C 库的 `snprintf/sprintf` 可直接完成整数转字符串。自定义实现增加了代码量和维护成本。但考虑到嵌入式环境可能库不完整，此做法也可以理解。

7. 魔数 (Magic Number) 较多

代码中存在如 80（滚动总列数）、48（字符总宽度）、-3（方块初始 Y 坐标）等硬编码数字，缺乏命名常量，阅读和维护时需推导其含义。

改进: 定义命名常量如 `#define SCROLL_TOTAL_COLS 80`

8. 可扩展性局限

当前游戏逻辑、LED 渲染、音效全耦合在一个 `tetris_thread` 中，如果想增加新显示设备（如 LCD）或新音效方案，需要改动核心代码。

改进: 可将显示和音效抽象为独立接口，游戏逻辑只调用接口